

DOCUMENTACIÓN TÉCNICA

Proxy inverso L7 de alto rendimiento Rust · hilos · epoll / kqueue

Arquitectura, módulos, protocolos, decisiones de diseño, pruebas, rendimiento y comparativa con las versiones en C.

Versión	2.0.0 — rama <code>main</code> de <code>gusuval/rust_mas_hilos_proxy</code>
Fecha	1 de octubre de 2026
Plataformas	Linux (epoll) verificada · macOS/BSD (kqueue) escrita, sin compilar ni ejecutar
Documentos	<code>SPEC.md</code> · <code>docs/requerimientos.md</code> · <code>docs/decisiones.md</code> · <code>docs/verificacion.md</code>

Índice

1. Visión general

- Alcance
- Historia del proyecto
- Cifras clave

2. Arquitectura

- Hilos
- Flujo de una petición
- Estructura del código

3. Event loop (`mio`)

- Registro y tokens con generación
- Edge-triggered y `sess_drive`
- Timers y reloj
- Liberación diferida

4. Master y workers

- Canal master → worker
- Sockets de escucha
- Señales, relanzamiento y parada

5. Ciclo de vida de una conexión

- Máquina de estados
- Buffers y contrapresión
- Timeouts

6. HTTP/1.1

- Parser
- Reescritura de cabeceras
- Cuerpos
- Keep-alive, pool y reintento
- Pipelining y WebSocket
- Errores generados por el proxy

7. TLS y SNI

8. Enrutamiento, balanceo y salud

9. Configuración y recarga en caliente

10. Observabilidad

11. Pruebas y rendimiento

12. Comparativas de las ejecuciones

- C: procesos (`fork`) frente a hilos (`pthread`)
- Rust frente a C
- Evolución de las tres versiones

13. Decisiones de diseño y limitaciones

1. Visión general

El proyecto es un **proxy inverso de capa 7** escrito en Rust sobre un event loop propio por hilo (`mio` , una capa fina sobre `epoll` y `kqueue`), sin runtime asíncrono. Recibe conexiones HTTP y HTTPS en uno o varios puertos, decide el destino por el nombre de dominio (`Host:`) y reparte las peticiones entre varios servidores backend.

1.1 Alcance

Incluido	Fuera de alcance
Linux (<code>epoll</code>) y macOS/BSD (<code>kqueue</code>)	Windows / IOCP
HTTP/1.0 y HTTP/1.1 en ambos lados	HTTP/2 y HTTP/3
Terminación TLS 1.2 / 1.3 (<code>rustls</code>) con SNI	TLS hacia los backends
Keep-alive, pipelining, chunked, WebSocket	Caché y compresión
Recarga automática y por <code>SIGHUP</code>	API de administración remota

1.2 Historia del proyecto

Versión	Lenguaje y modelo	Repositorio
1.0	C11 + Meson; master + N procesos worker (<code>fork</code>)	<code>1.5.10-proxy-epoll-kqueue</code>
1.1	C11 + Meson; master + N hilos worker (<code>pthread</code>)	<code>version_pthread_proxy</code>
2.0	Rust + cargo ; master + N hilos worker (<code>std::thread</code> + <code>mio</code>)	<code>rust_mas_hilos_proxy</code> (este documento)

Las tres versiones tienen el mismo comportamiento: la batería de integración escrita para la versión en C pasa entera contra el binario de Rust. El capítulo 12 compara su rendimiento.

1.3 Cifras clave

202.156 req/s HTTPS (100 conexiones)	341.340 req/s HTTP (200 conexiones)	0 errores en 27 M peticiones	103 + 43 comprobaciones de integración + tests unitarios
--	---	--	--

La meta del enunciado (≥ 50.000 req/s sobre HTTPS) se supera entre 3,2× y 4× en todos los escenarios medidos (capítulo 11).

2. Arquitectura

2.1 Hilos

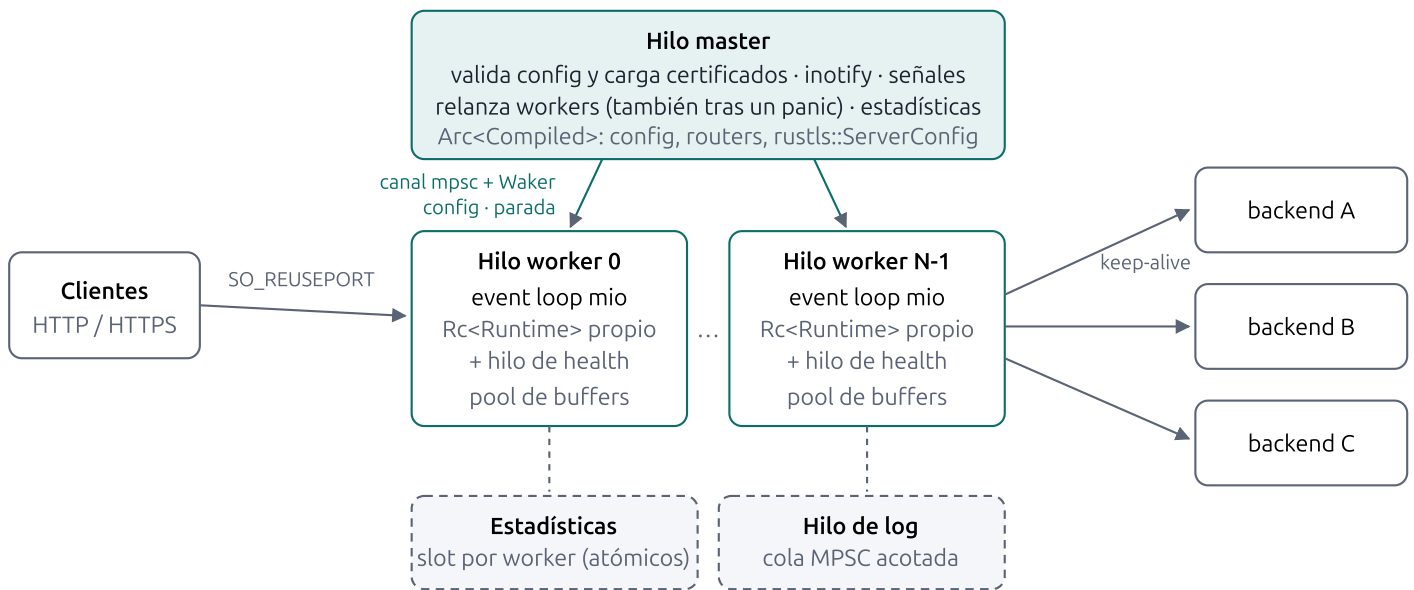


Figura 1. Un proceso: hilo master, N hilos worker (cada uno con su event loop) y los hilos auxiliares.

El **master** no atiende tráfico. Valida la configuración y la *compila* una sola vez (routers y contextos TLS con los certificados cargados), crea los hilos worker, les envía cada generación por un canal, vigila el fichero, atiende las señales, relanza los workers que terminan y sirve las estadísticas. Cada **worker** ejecuta un event loop y tiene un hilo auxiliar de sondas de salud; un único hilo de log da servicio a todo el proceso.

La regla que hace correcto el diseño la impone el compilador: lo que se comparte entre hilos (la generación de configuración, la salud de cada servidor, las estadísticas) va en `Arc` con tipos `Sync`; el estado que cambia en cada petición (pools, contadores, balanceo) vive en `Rc / Cell`, que no son `Send`, así que no puede escaparse del hilo por error.

2.2 Flujo de una petición

1. El kernel entrega la conexión a un worker (`SO_REUSEPORT` en Linux).
2. Si el frontend es TLS, se completa el handshake y el resolutor elige el certificado por SNI.
3. Se parsea la cabecera en cuanto está completa y se busca la ruta por `Host`: : exacta → wildcard → `default`.
4. La estrategia del backend elige un servidor sano. Se reutiliza una conexión del pool keep-alive o se abre una nueva.
5. Se reenvía la cabecera reescrita (`X-Forwarded-*`, sin hop-by-hop) y el cuerpo en streaming.
6. La respuesta vuelve reescrita (sin `X-Backend-Load`). Al terminar, la conexión al backend vuelve al pool y la del cliente espera la siguiente petición.

2.3 Estructura del código

Módulo	Responsabilidad
<code>master</code> , <code>main</code>	Arranque, hilos worker, recarga, señales (self-pipe), inotify y socket de estadísticas
<code>worker</code>	Event loop <code>mio</code> , listeners, swap de generaciones, tick y parada ordenada
<code>conn</code>	Máquina de estados cliente↔upstream, TLS, pool keep-alive, reintentos y timeouts
<code>http</code>	Parser HTTP/1.x incremental y delimitación de cuerpos
<code>tls</code>	Resolvedor de certificado por SNI y <code>rustls::ServerConfig</code>
<code>router</code>	Tabla hash djb2 de dominios exactos, wildcards por sufijo más largo, default
<code>balancer</code>	Servidores, estrategias de balanceo y estado de salud atómico
<code>health</code>	Sondas TCP/HTTP en un hilo por generación
<code>runtime</code>	<code>Compiled</code> (compartida) y <code>Runtime</code> (del worker) con refcount (RCU)
<code>config</code>	Parser TOML (crate <code>toml</code>) con validación estricta
<code>log</code>	Cola MPSC acotada de 4096 líneas y hilo consumidor
<code>stats</code>	Slots por worker y JSON agregado
<code>buffer</code> , <code>timer</code> , <code>slab</code>	Pool de slots de 16 KB, min-heap perezoso, slab con generaciones

Todo se compila como la biblioteca `proxy` , que reutilizan el binario del proxy y las herramientas de prueba `test_backend` y `ws_probe` (`src/bin/`). Son unas 7.100 líneas de Rust. `unsafe` solo aparece en `master.rs` , en las llamadas al sistema de señales e inotify.

3. Event loop (`mio`)

3.1 Registro y tokens con generación

Cada hilo worker tiene su `mio::Poll` (epoll en Linux, kqueue en macOS/BSD). Sesiones, upstreams y listeners viven en slabs, y el token que se registra en el `Poll` (y en los timers) codifica tres campos:

```
token = clase (2 bits) | generación (30 bits) | índice en la slab (32 bits)
// clases: 0 = control (Waker, tick), 1 = listener, 2 = sesión, 3 = upstream
```

Cuando se libera un hueco de la slab y se reutiliza, su generación cambia: un evento o timer tardío del objeto anterior ya no casa y se ignora. Es la versión segura del "marcar cerrado e ignorar eventos tardíos" de la versión en C.

3.2 Edge-triggered y `sess_drive`

Todos los sockets se registran una sola vez, con lectura y escritura, en modo **edge-triggered** (`EPOLLET` / `EV_CLEAR` , el modo de `mio`). El kernel avisa una vez por cambio de estado, así que cada sesión lleva dos flags por socket: `rd` y `wr` ("el último intento no terminó en `WouldBlock` ").

```
sess_drive(s):
  repetir (máx. 32 vueltas):
    cli_read      // socket cliente (o rustls) → inb, hasta WouldBlock o buffer lleno
    process       // parsear, enrutar, mover cuerpo inb → uo
    up_io         // connect, uo → backend, backend → ui
    process       // respuesta ui → out
    cli_write     // out → socket cliente (o rustls)
  mientras haya progreso
  si se agota el presupuesto: replanificar al final de la iteración
```

Como se repite hasta que no hay progreso, nunca queda trabajo pendiente sin un evento que lo despierte, el error clásico con edge-triggered. El presupuesto de 32 vueltas evita que una transferencia grande acapare el loop.

3.3 Timers y reloj

Los timers viven en un min-heap. Cada objeto guarda su `TimerState` (deadline pedido y clave en el heap). Programar es **perezoso**: retrasar un deadline solo actualiza un campo, y el timer se reubica cuando vence su clave antigua; adelantarlo inserta una entrada nueva y la vieja queda obsoleta. Así, refrescar un timeout con cada byte recibido cuesta $O(1)$.

El reloj (`Worker::now`) se cachea por iteración y se actualiza justo después de `poll` , antes de despachar. En la primera versión en C se actualizaba después y, tras un periodo ocioso, los handlers calculaban deadlines ya vencidos (408/504 espurios).

3.4 Liberación diferida

Un mismo lote de eventos puede traer varios de una sesión que se cierra al procesar el primero, y las funciones de la misma pila de llamadas aún la referencian por índice. Al cerrarla se quita del `Poll` y se marca `closed` ; el hueco de la slab se libera al final de la iteración, junto con las sesiones replanificadas.

4. Master y workers

4.1 Canal master → worker

Cada worker tiene un canal `mpsc` y un `mio::Waker` registrado en su `Poll`. El master envía un mensaje y despierta al worker:

```
enum WorkerMsg {
    Config(Arc<Compiled>), // nueva generación de configuración
    Stop,                  // parada ordenada
    Panic,                 // solo builds de depuración: prueba de relanzamiento
}
```

El master construye el `poll` del worker y le deja la configuración inicial en el canal antes de lanzar el hilo, así que el worker la encuentra al arrancar. Envía la **generación ya compilada** en lugar de pedir a cada worker que relea el fichero: todos aplican exactamente la misma versión y los certificados se leen una sola vez.

En sentido contrario, cada hilo lleva un guardia (`ExitGuard`) que, al soltarse, marca su slot de estadísticas, envía su número al master por otro canal y lo despierta con el `Waker` del master. El guardia se suelta también durante el *unwinding* de un `panic`.

4.2 Sockets de escucha

Plataforma	Estrategia	Motivo
Linux	Cada worker abre sus sockets con <code>SO_REUSEPORT</code> (<code>socket2</code>)	El kernel reparte las conexiones por hash, sin <i>thundering herd</i>
macOS/BSD	El master abre los sockets, los incluye en la generación y cada worker usa un duplicado	Allí <code>SO_REUSEPORT</code> no reparte carga

En una recarga, cada worker sincroniza sus listeners con los frontends nuevos: conserva los existentes, abre los añadidos y cierra los eliminados.

4.3 Señales, relanzamiento y parada

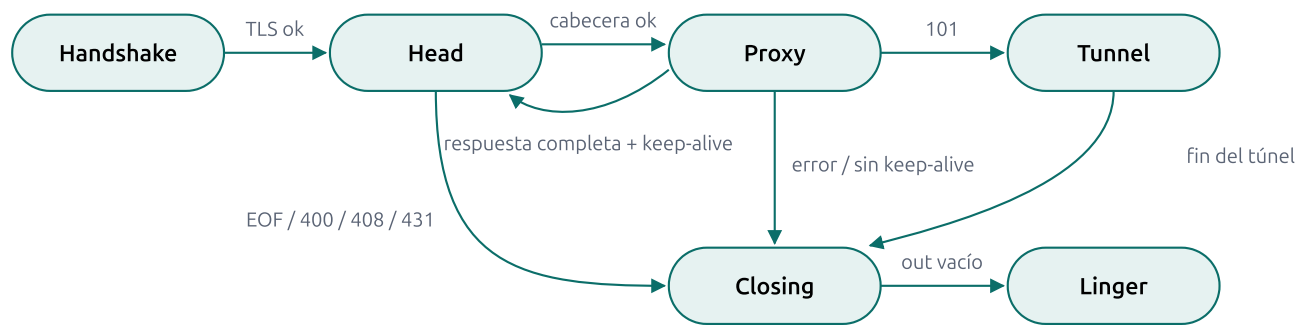
Evento	Master	Worker
<code>SIGHUP</code>	Recarga forzada	Bloqueada (la coordina el master)
Cambio en el fichero	Recarga tras 200 ms de debounce, solo si el contenido cambió	—
<code>SIGTERM</code> / <code>SIGINT</code>	Envía <code>Stop</code> a cada worker y espera hasta 12 s	Cierra listeners y conexiones ociosas; espera hasta 10 s a las activas
Fin de un hilo worker	<code>join</code> y relanzamiento con backoff; abandona tras 10 fallos seguidos al arrancar	—
<code>SIGUSR1</code> (depuración)	Envía <code>Panic</code> a un worker	<code>panic!</code> : termina solo ese hilo

Las señales se procesan con el *self-pipe trick*: el handler (`sigaction`) solo escribe el número de señal en un pipe que vigila el `poll` del master. Los workers y sus hilos de health se crean con todas las señales bloqueadas (`pthread_sigmask`), así que solo las recibe el master.

Recuperación tras un fallo. Con el perfil `panic = "unwind"`, un `panic` en un worker solo termina ese hilo: sus sockets se cierran al soltarse, el master lo relanza y el tráfico sigue (lo comprueba la batería de integración). En la versión C con hilos esto no era posible: cualquier fallo de un worker tumbaba el proceso.

5. Ciclo de vida de una conexión

5.1 Máquina de estados



Linger: close_notify (TLS), shutdown(Write) y descartar la entrada hasta EOF o 2 s, para que un RST no se coma la respuesta.

Figura 2. Estados de una sesión de cliente (`SState` en `conn.rs`).

5.2 Buffers y contrapresión

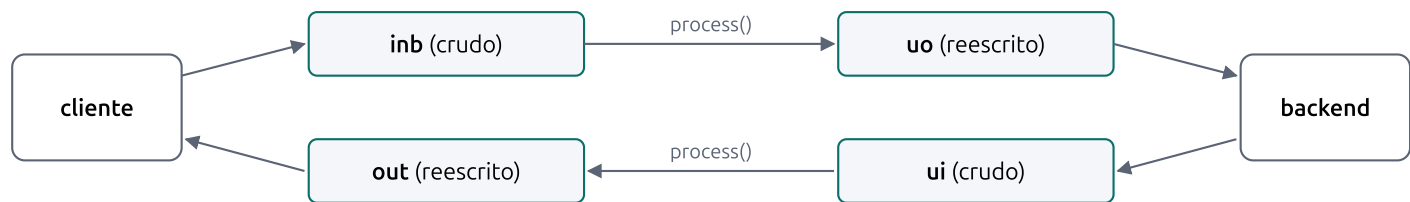


Figura 3. Los cuatro buffers de 16 KB de una sesión.

Los buffers salen de un pool de slots por hilo y se piden solo cuando hacen falta: una conexión keep-alive ociosa no retiene ninguno. La contrapresión sale sola. Si `out` se llena porque el cliente lee despacio, `process` deja de vaciar `ui`, se deja de leer del backend y la ventana TCP frena al servidor. En TLS, el texto plano pendiente de cifrar en rustls se limita a 64 KB por conexión para no romper esa cadena. Con 10 MB de subida, el pico de memoria de todo el proceso se mantiene muy por debajo del límite que comprueban los tests (2×64 MB).

5.3 Timeouts

Estado	Deadline	Al vencer
Handshake	aceptación + <code>client_header</code>	cerrar
Head, sin datos	entrada en el estado + <code>client_idle</code>	cerrar (keep-alive ocioso)
Head, cabecera a medias	primer byte + <code>client_header</code>	408
Proxy, conectando	inicio del connect + <code>upstream_connect</code>	fallo pasivo; otro servidor, o 504
Proxy	último progreso + <code>upstream_read</code>	504 (o cerrar si ya hay respuesta a medias)
Tunnel	último progreso + <code>client_idle</code>	cerrar
Closing / Linger	5 s / 2 s	cerrar

6. HTTP/1.1

6.1 Parser

La cabecera se busca como `\r\n\r\n`, recordando hasta dónde se ha escaneado, y se parsea de una vez cuando está completa. Los campos son rangos (`Span`) sobre el buffer, sin copias. Así se tolera cualquier fragmentación (hay tests que alimentan el parser byte a byte y en trozos aleatorios) sin reescanear. Validaciones:

- Método como *token*, versión exactamente `HTTP/1.0` o `HTTP/1.1`, un único espacio entre elementos.
- Nombres de cabecera como *token*, sin espacio antes de `:`. Se rechazan el *obs-fold* y los caracteres de control.
- `Host` obligatorio en 1.1 y único; se le quita el puerto.
- **Anti request smuggling:** `Content-Length` + `Transfer-Encoding` → 400; varios CL distintos → 400; TE sin `chunked` final → 400.
- Cabecera de más de 16 KB o más de 100 cabeceras → 431.

6.2 Reescritura de cabeceras

Hacia el backend	Hacia el cliente
Línea de petición reescrita a <code>HTTP/1.1</code> <code>X-Forwarded-For</code> : se añade la IP (concatenando) <code>X-Forwarded-Proto</code> y <code>X-Real-IP</code> : se reemplazan Sin hop-by-hop (<code>Connection</code> , <code>Keep-Alive</code> , <code>TE</code> , <code>Trailer</code> , <code>Upgrade</code> ... y las nombradas en <code>Connection</code>)	Sin hop-by-hop ni <code>X-Backend-Load</code> <code>Connection: close</code> si se va a cerrar <code>Connection: keep-alive</code> para clientes 1.0 persistentes En un <code>101</code> se conservan <code>Connection</code> y <code>Upgrade</code>

Nunca se eliminan `Host`, `Content-Length` ni `Transfer-Encoding`, aunque aparezcan en `Connection: :` si se pudiera forzar al proxy a quitar el encuadre, eso sería un vector de smuggling.

6.3 Cuerpos

`http::Body` solo sigue los límites del mensaje. Los bytes se reenvían tal cual, sin decodificar ni almacenar.

Modo	Cuándo
<code>None</code>	Sin cuerpo; <code>HEAD</code> , <code>1xx</code> , <code>204</code> , <code>304</code>
<code>Length</code>	<code>Content-Length</code>
<code>Chunked</code>	Máquina de 9 estados: tamaño hex, extensiones, datos, CRLF y trailers
<code>Eof</code>	Respuesta sin longitud: se reenvía hasta el cierre y después se cierra también el cliente

6.4 Keep-alive, pool y reintento

- **Cliente:** persistente por defecto en 1.1; en 1.0 solo con `Connection: keep-alive`.
- **Backend:** pool LIFO de conexiones ociosas por servidor (`max_idle`, `upstream_idle`). Una conexión vuelve al pool si la respuesta terminó limpia, el backend no pidió cerrar y no sobran bytes. Mientras está en el pool, cualquier dato o cierre la invalida (se confirma con `peek` para descartar eventos tardíos).
- **Reintento único:** si una conexión *reutilizada* falla antes del primer byte de respuesta y la petición es idempotente y sin cuerpo, se reenvía por una conexión nueva. Para eso se conserva la cabecera ya enviada en `uo`. Un POST nunca se reintenta.
- Si el connect falla, se prueba con otro servidor (hasta 3 intentos, excluyendo los ya probados).

6.5 Pipelining y WebSocket

Las peticiones pipelined se leen por adelantado hasta llenar `inb`, pero se despachan en serie: la siguiente empieza cuando la respuesta anterior se ha leído entera del backend. Así el orden queda garantizado sin colas, y cada petición puede ir a un backend distinto.

Si la petición trae `Connection: Upgrade` y el backend responde `101`, la sesión pasa a **Tunnel**: copia bytes en ambos sentidos hasta que un lado cierra. La conexión nunca vuelve al pool.

6.6 Errores generados por el proxy

Código	Causa
400	Petición malformada, smuggling, chunked inválido, 1.1 sin <code>Host</code>
404	Ninguna ruta (ni <code>default</code>) para el <code>Host</code>
408	Cabecera incompleta tras <code>client_header</code>
421	SNI y <code>Host</code> distintos, y el certificado servido no cubre el <code>Host</code>
431	Cabecera de más de 16 KB
502	Backend inalcanzable tras los intentos, respuesta inválida o conexión rota
503	Ningún servidor sano en el backend, o frontend eliminado en una recarga
504	Sin datos del backend durante <code>upstream_read</code>

Todas estas respuestas llevan `Connection: close`.

7. TLS y SNI

Cada frontend TLS tiene un `rustls::ServerConfig` (TLS 1.3 y 1.2, preferencia de cifrados del servidor) con un resolutor propio (`ResolvesServerCert`). El resolutor recibe el `ClientHello` y elige el certificado en este orden:

1. Coincidencia exacta con el patrón `sni` de la config.
2. Wildcard de **una sola etiqueta** (RFC 6125): `*.example.com` cubre `a.example.com`, no `a.b.example.com`. Gana el patrón más largo.
3. Si no hay coincidencia, `default_cert`.

Al cargar cada certificado, `CertifiedKey::from_der` comprueba que la clave corresponde al certificado. Tras el handshake, la sesión guarda el nombre SNI y el patrón servido: si el cliente envió SNI, el `Host` es distinto y el patrón no lo cubre → `421 Misdirected Request`.

La API de `rustls` no hace E/S: se le dan bytes cifrados (`read_tls + process_new_packets`), se le pide texto plano (`reader`), se le da texto plano (`writer`) y se le piden bytes cifrados (`write_tls`). Eso encaja directamente con los flags `rd / wr` del event loop, sin los casos cruzados de OpenSSL (una lectura que necesita escribir). El proveedor criptográfico se elige al compilar: `aws-lc-rs` (por defecto) o `ring`.

8. Enrutamiento, balanceo y salud

8.1 Enrutamiento

Cada frontend tiene su tabla, inmutable y compartida entre hilos: hash djb2 con direccionamiento abierto para dominios exactos, lista de wildcards ordenada por longitud de sufijo (el más largo gana; aquí sí cubren varias etiquetas) y ruta `default` opcional. Todas las búsquedas son insensibles a mayúsculas.

8.2 Estrategias de balanceo

Estrategia	Algoritmo
<code>round_robin</code>	Siguiente servidor sano en orden
<code>weighted</code>	<i>Smooth weighted round robin</i> (nginx): con pesos 3:1 reparte 75/25 sin ráfagas largas
<code>least_conn</code>	Menos peticiones activas en este worker; si empatan, rotación
<code>least_load</code>	Carga de <code>X-Backend-Load</code> suavizada con EMA ($\alpha = 0,3$), que caduca a los 5 s. Se eligen dos candidatos al azar (<i>power of two choices</i>) y gana el de menor carga. Si no hay carga fresca, decide <code>least_conn</code> ; si también empatan, gana el que no tiene carga conocida (exploración)

El estado de balanceo de cada servidor (activas, carga, peso actual, contadores) está en `cell` dentro del `Runtime` de su worker: se modifica sin locks y el compilador garantiza que no lo toca otro hilo.

8.3 Salud

- **Pasiva** (hilo del loop): connect rechazado o reset antes de la respuesta cuentan como fallo; tras `fall` fallos seguidos, el servidor pasa a `down`. Sin sondas activas, pasado `interval` se reintenta en modo *half-open*: basta un fallo para volver a `down`.
- **Activa** (hilo propio por generación y worker): TCP connect o `GET path` esperando `2xx`, cada `interval`; tras `rise` éxitos, `up`.
- El estado (`up`, `fails`, `oks`, `down_since`) está en un `Arc<ServerShared>` con atómicos: es lo único que comparten los dos hilos.

9. Configuración y recarga en caliente

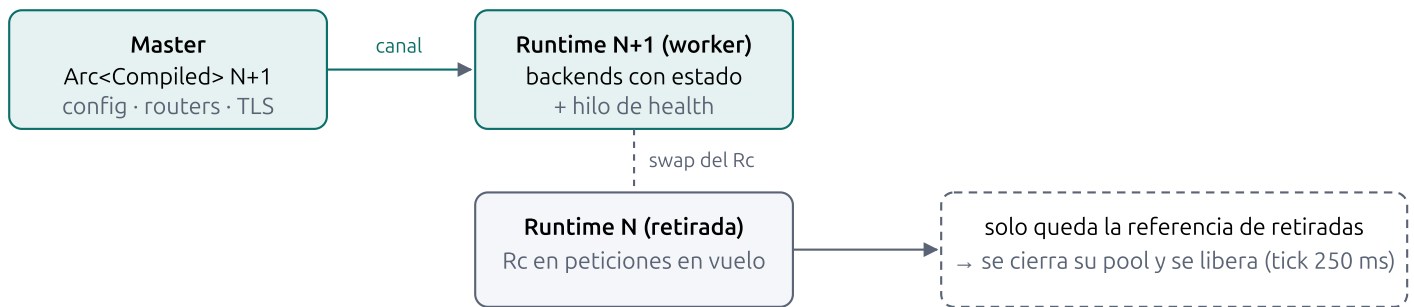


Figura 4. Generaciones de configuración: compartida (Arc) en el master y por worker (Rc) con refcount, sin atómicos en el hilo del loop.

1. **Disparo:** inotify sobre el *directorio* (cubre la escritura in situ y el rename de `vim / sed -i`) o `SIGHUP`. Debounce de 200 ms. Si el contenido no cambió, no se hace nada (salvo con `SIGHUP`). Fuera de Linux se sondea el fichero cada 500 ms.
2. **Compilación en el master:** TOML, reglas de coherencia, routers y carga de certificados. Si algo falla, se registra el error y se mantiene la configuración actual.
3. **Envío** del `Arc<Compiled>` a cada worker, que crea su `Runtime` (backends con estado, hilo de health), sincroniza sus listeners y cambia el `Rc`.
4. Las peticiones en vuelo terminan con su generación; en las conexiones keep-alive, la petición siguiente ya usa la nueva. El hilo de health de la generación vieja recibe la orden de parar y termina por su cuenta: todo lo que usa lo tiene por `Arc`.

La validación es estricta: una clave desconocida (`workerz`) es un error. Así una errata no se ignora en silencio.

10. Observabilidad

10.1 Log

Un único logger para todo el proceso: los productores (master, workers, hilos de health) formatean la línea (hasta 512 B) y la dejan con `try_send` en una cola MPSC acotada de 4096 líneas; un hilo consumidor las vuelca por lotes. Si la cola está llena, la línea se descarta y se incrementa `log_dropped`: el loop nunca se bloquea. Cada hilo tiene su etiqueta (`master` , `w0` , `w1` ...), que heredan sus hilos de health.

10.2 Estadísticas

Cada worker tiene un slot alineado a 64 B (contra el *false sharing*) con contadores atómicos (*relaxed*) y una tabla de servidores que publica cada 250 ms bajo un `Mutex` . El master agrega todo y sirve JSON por el socket UNIX `stats_socket` . Un servidor aparece como `degraded` cuando los workers discrepan sobre su salud.

11. Pruebas y rendimiento

11.1 Estrategia de pruebas

Nivel	Contenido	Casos
Unitario (cargo test)	Parser (fragmentación, límites, smuggling, chunked), router, SNI, balanceo (4 estrategias, salud), config (19 reglas), log (cola llena y concurrente), timers, slab, buffers, estadísticas	43
Integración	12 backends reales y el proxy: enrutamiento, errores, cabeceras, cuerpos de 10 MB, keep-alive, pool, reintento, pipelining, timeouts, WebSocket, TLS/SNI, balanceo, failover, recarga, stats, hilos, relanzamiento tras un panic , carga con recargas y parada	103
Estático	cargo clippy --all-targets y cargo fmt --check	0 avisos

La batería de integración es la misma de la versión en C (99 comprobaciones), más 4 de relanzamiento tras un panic que solo existen en el build de depuración. Se lanza con tests/integration/run.sh o con cargo test -- --ignored . Herramientas propias: test_backend (HTTP/1.1 y WebSocket sobre mio , con latencia, carga y descarte de conexiones configurables) y ws_probe (cliente WebSocket con TLS).

11.2 Benchmark

Escenario (wrk, 30 s)	Protocolo	Req/s	Latencia media	p99	Errores
-t4 -c100	HTTPS	202.156	0,41 ms	1,08 ms	0
-t4 -c200	HTTPS	189.936	0,72 ms	2,10 ms	0
-t4 -c400	HTTPS	163.825	1,53 ms	5,46 ms	0
-t4 -c200	HTTP	341.340	0,56 ms	1,76 ms	0

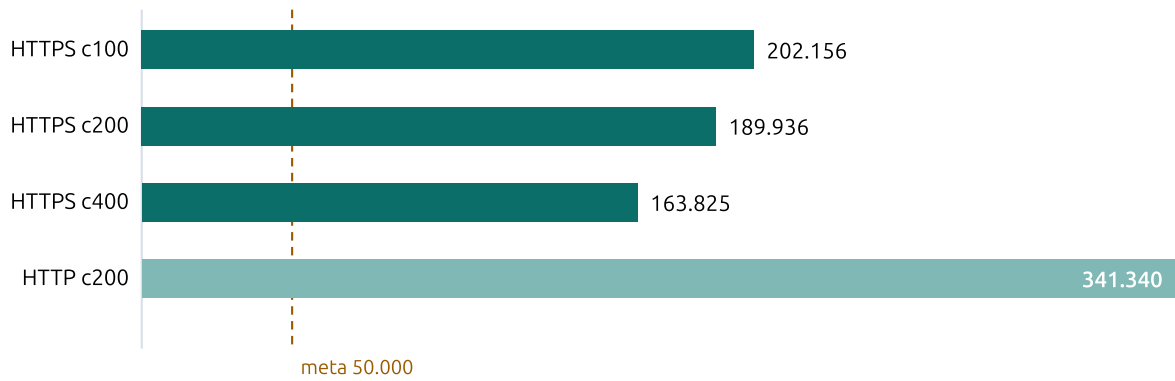


Figura 5. Peticiones por segundo del stack completo en Rust (proxy y backends) frente a la meta de 50.000.

Entorno: AMD Ryzen 7 7735HS (8 núcleos / 16 hilos) en WSL2, kernel 6.18. Build release con LTO y codegen-units = 1 , rustls 0.23 con aws-lc-rs , 8 workers, access log desactivado, 2 test_backend de 2 hilos y respuesta de 100 B. Afinidad con taskset : wrk en las CPUs 0-3, el proxy en 4-11 y los backends en 12-15. En esa ejecución hubo 26,95 M peticiones, todas 2xx, servidas con solo 845 conexiones a los backends gracias al pool. Como wrk, proxy y backends comparten máquina, las cifras son una cota inferior. Una segunda ejecución dio 201.168 / 193.383 / 163.089 / 335.182 req/s.

12. Comparativas de las ejecuciones

Todas las mediciones usan el mismo script (`bench/bench_proxy.sh`), la misma máquina y la misma configuración que el capítulo anterior (8 workers, 2 backends de 2 hilos, respuesta de 100 B, `wrk -t4` , 30 s por escenario, CPUs fijadas). Se hicieron el 1 de octubre de 2026, alternando versiones para que la carga de la máquina afectara a todas por igual. Los informes completos están en `bench/results/` .

12.1 C: procesos (`fork`) frente a hilos (`pthread`)

Primer cambio de modelo: la versión en C pasó de master + N *procesos* worker a un proceso con N *hilos* worker. Dos ejecuciones alternas de cada versión, con el backend en C:

Escenario	fork #1	fork #2	pthread #1	pthread #2
HTTPS -c100	218.432	228.312	224.999	228.962
HTTPS -c200	200.599	220.085	219.959	206.081
HTTPS -c400	26.663 [⚠]	173.506	168.219	174.086
HTTP -c200	297.652	317.289	336.447	338.965

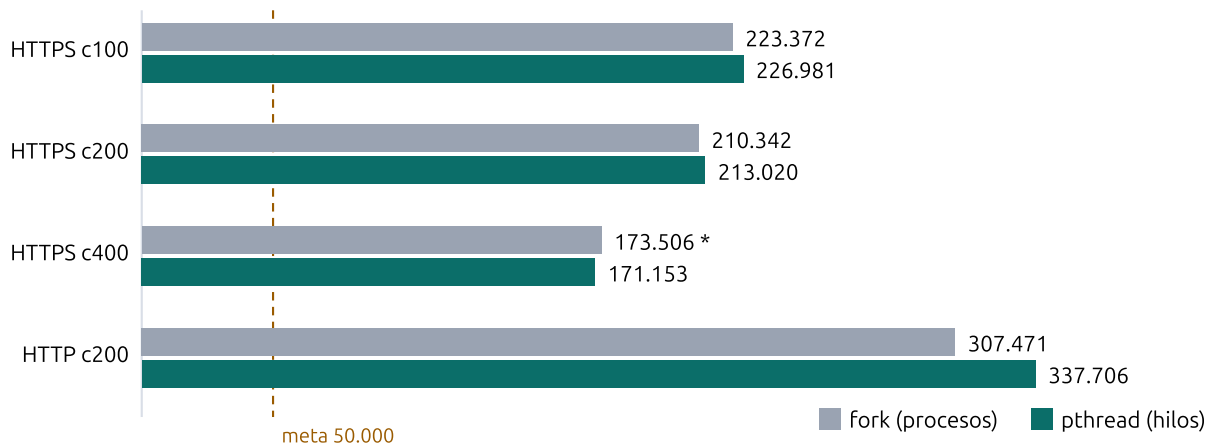


Figura 6. Media de req/s por versión de C (* fork c400: solo la ejecución #2).

- **HTTPS:** equivalentes. Las diferencias (± 5 %) son del tamaño de las que hay entre dos ejecuciones de la misma versión.
- **HTTP:** los hilos salen un 6-13 % por encima en las dos ejecuciones (un solo espacio de direcciones: menos presión de TLB y caché que 8 procesos).
- [⚠] **fork #1, c400:** wrk tardó 2,51 min en vez de 30 s y terminó con 400 timeouts (uno por conexión); durante la prueba iba a unos 134k req/s. No se reprodujo y no tiene causa identificada.

12.2 Rust frente a C

Para aislar el proxy, las tres primeras columnas usan el mismo backend (el `test_backend` en C). La última es el stack completo en Rust.

Escenario	C + OpenSSL	Rust + ring	Rust + aws-lc	Rust, backend Rust
HTTPS -c100	228.532	201.241	203.985	201.168 / 202.156
HTTPS -c200	220.261	195.717	197.585	193.383 / 189.936
HTTPS -c400	177.788	163.420	162.959	163.089 / 163.825
HTTP -c200	348.992	359.452	365.279	335.182 / 341.340
p99 HTTPS -c100	1,32 ms	1,02 ms	1,29 ms	1,01 / 1,08 ms

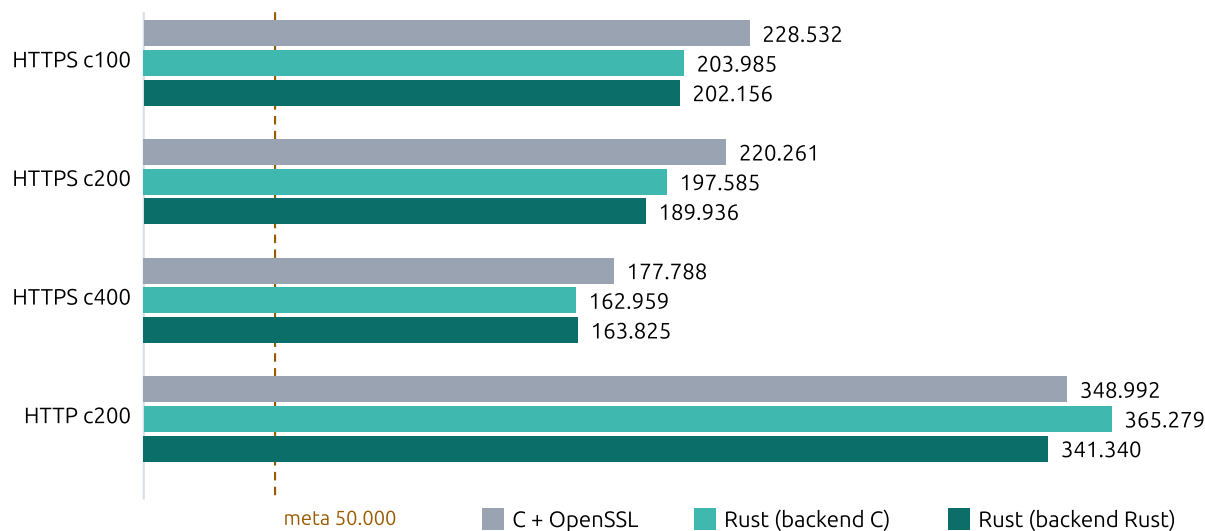


Figura 7. Req/s del proxy en C y en Rust (rustls con aws-lc-rs).

- **HTTP:** con el mismo backend, el proxy en Rust iguala o supera al de C (359-365k frente a 349k). La parte propia del proxy (event loop, parser, buffers, pool) no es más lenta que en C.
- **HTTPS:** el proxy en Rust queda un 8-12 % por debajo. Como en HTTP no hay diferencia, la causa está en la capa TLS. Cambiar el proveedor de `ring` a `aws-lc-rs` no la mueve, así que no es el cifrado AES-GCM. La explicación más probable es el coste por registro de rustls (buffers internos con memoria dinámica y una copia más del texto plano), que pesa mucho con respuestas de 100 B. Un allocator más rápido (`mimalloc`) solo dio un 2-3 %, dentro del ruido, y se descartó.
- **Backend de pruebas:** la primera versión del `test_backend` en Rust ponía a cero un buffer de 64 KB en cada evento y limitaba el escenario HTTP a ~298k req/s. Con un buffer reutilizado por hilo llega a 335-341k.

12.3 Evolución de las tres versiones

Escenario (media)	C, fork	C, pthread	Rust, hilos	Rust / C pthread
HTTPS -c100	223.372	227.498	201.662	-11 %
HTTPS -c200	210.342	215.434	191.660	-11 %
HTTPS -c400	173.506	173.364	163.457	-6 %
HTTP -c200	307.471	341.468	338.261	-1 %
Ejecuciones promediadas	2 (c400: 1)	3	2	

Las versiones en C usan el backend en C; la de Rust, el backend en Rust. Las tres superan la meta de 50.000 req/s por HTTPS más de 3 veces, sin errores. A cambio de ese ~10 % en HTTPS, la versión en Rust aporta seguridad de memoria en todo el camino de datos, el relanzamiento de workers tras un `panic` y la carga única de los certificados. Si hiciera falta recuperar el rendimiento de TLS, las opciones serían la API *unbuffered* de rustls (cifrar sobre los buffers propios, sin copias intermedias) o usar OpenSSL desde Rust.

13. Decisiones de diseño y limitaciones

Decisión	Alternativa descartada	Motivo
<code>mio</code> y un event loop por hilo	<code>tokio</code> (runtime con <i>work stealing</i>)	El enunciado pide programar contra <code>epoll/kqueue</code> ; el port es fiel y la comparación mide lenguaje y TLS, no dos arquitecturas
<code>rustls</code>	OpenSSL desde Rust	TLS en Rust seguro y API sin E/S que encaja con el loop; a cambio, ~10 % menos en HTTPS
<code>Arc</code> para lo compartido, <code>Rc / Cell</code> para el estado del worker	Todo en <code>Arc</code> + atómicos	Sin sincronización en el camino caliente; el compilador impide compartir el estado local
Slab con generaciones	Punteros / <code>Rc<RefCell></code> por conexión	Tokens seguros frente a eventos tardíos, sin ciclos de referencias
Hilos (master + workers)	Procesos con <code>fork</code>	Un solo proceso, menos memoria; el <code>panic</code> de un worker se recupera igualmente
ET con un único registro por socket	LT, o re-registro por cambio de estado	Menos syscalls; <code>sess_drive</code> evita perder eventos
Cabecera completa + delimitación de cuerpo aparte	Parser byte a byte con estado global	Más simple; sin copias ni reescaneo
Pipelining en serie	Varias peticiones en vuelo	Orden garantizado sin colas de respuesta
Reintento solo idempotente, sin cuerpo y en conexión reutilizada	Reintentar cualquier fallo	No duplicar operaciones con efectos
El master compila la config y la comparte	Cada worker relee el fichero	Todos aplican la misma versión; certificados leídos una vez

13.1 Limitaciones conocidas

- macOS/BSD: el código específico (sockets del master, sondeo del fichero) está escrito, pero no se ha compilado ni ejecutado en un Mac.
- HTTPS con respuestas pequeñas: ~10 % por debajo de la versión en C con OpenSSL (capítulo 12).
- Un cliente HTTP/1.0 recibe tal cual las respuestas chunked del backend.
- Cambiar `workers` requiere reiniciar.
- El timeout `upstream_connect` no tiene test automático: en loopback un connect rechazado falla al instante.
- Un fallo que no sea un `panic` (p. ej. un abort) seguiría terminando todo el proceso.